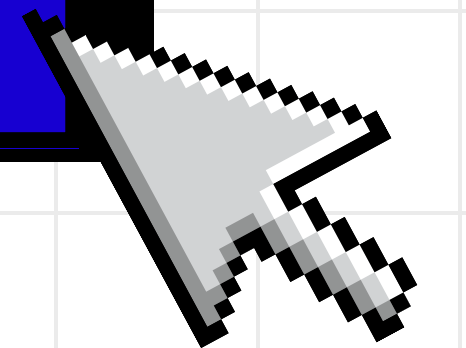
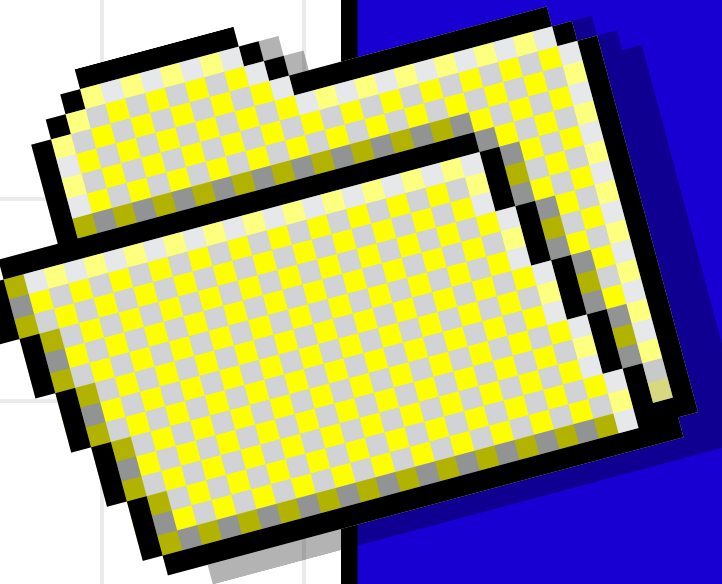
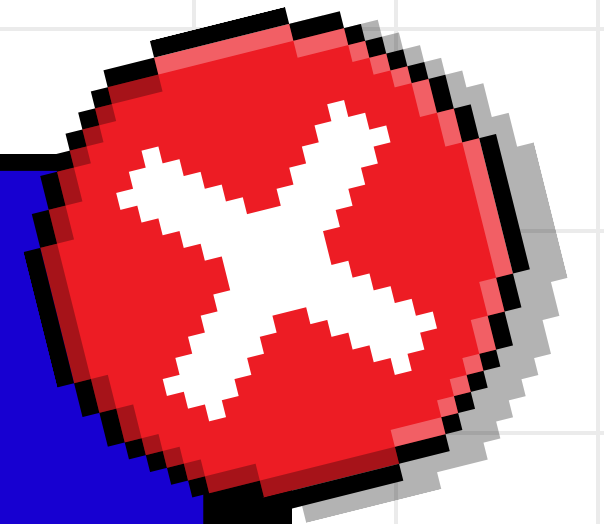
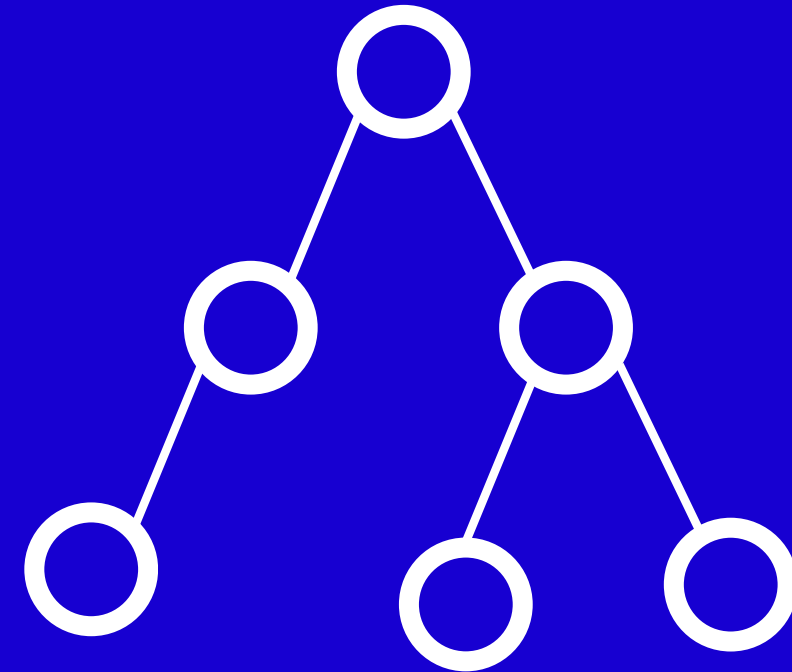


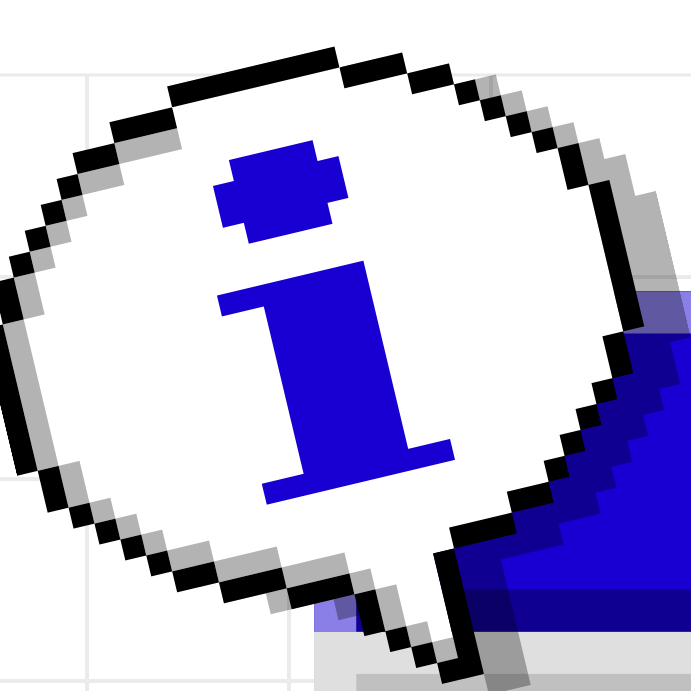
Algoritmos ordenamiento

HEAP SORT

Estudiantes:
Frabizio Arias Solano
Alejandro Baires Pérez

Grupo:12





¿QUÉ ES HEAP SORT?

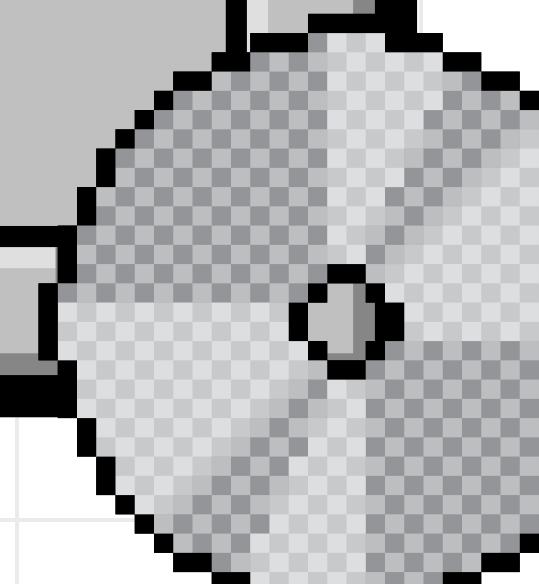
Heap Sort es un algoritmo de ordenamiento que utiliza una estructura de datos llamada Heap.

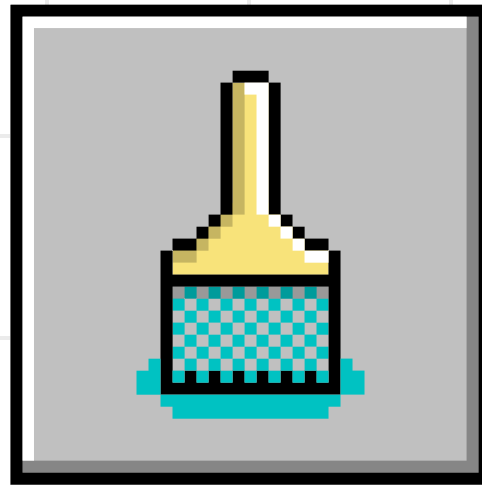
En este ejercicio se utiliza un Max Heap

Heap Sort funciona principalmente en dos partes:

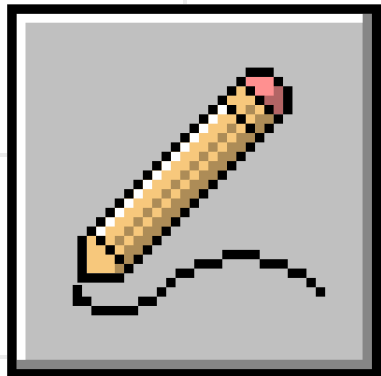
1. Construir un Max Heap.
2. Extraer repetidamente el elemento máximo y colocarlo al final del arreglo.

Complejidad: $O(n \log n)$

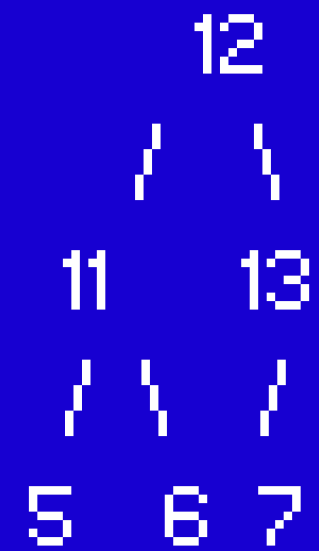




QUÉ ES UN MAX HEAP



Lista inicial:
[12, 11, 13, 5, 6, 7]



No es un Max Heap, porque:

$12 < 13$

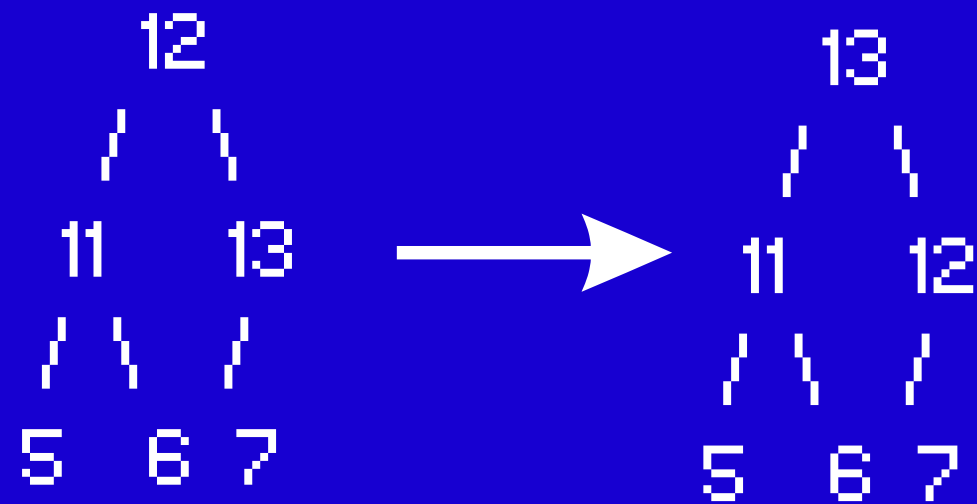
El elemento mayor debe estar en la raíz.

Regla del Max

Heap:

Padre $\geq$ Hijos

APLICAR HEAPIFY



Max Heap construido
[13, 11, 12, 5, 6, 7]

```
def heapify(arr, n, i):
    """
    Mantiene la propiedad de Max Heap en el subárbol
    cuyo nodo raíz está en la posición i.
    """
    mayor = i
    izquierdo = 2 * i + 1
    derecho = 2 * i + 2

    # Si el hijo izquierdo es mayor que la raíz
    if izquierdo < n and arr[izquierdo] > arr[mayor]:
        mayor = izquierdo

    # Si el hijo derecho es mayor que el mayor actual
    if derecho < n and arr[derecho] > arr[mayor]:
        mayor = derecho

    # Si el mayor no es la raíz
    if mayor != i:
        arr[i], arr[mayor] = arr[mayor], arr[i]

    # Aplicar heapify nuevamente
    heapify(arr, n, mayor)
```

PRIMERA EXTRACCION

Tenemos:

[13, 11, 12, 5, 6, 7]

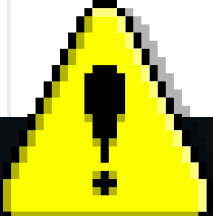
Pasamos:

[12, 11, 7, 5, 6, 13]

El 13 ya está ordenado y queda fuera del Heap.
Ahora aplicamos nuevamente heapify():

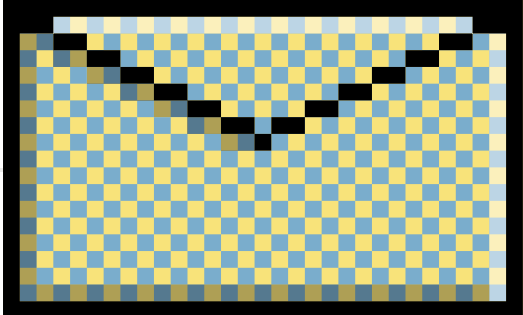


Resultado de primera extracción sin max heap:
[7, 11, 12, 5, 6, 13]

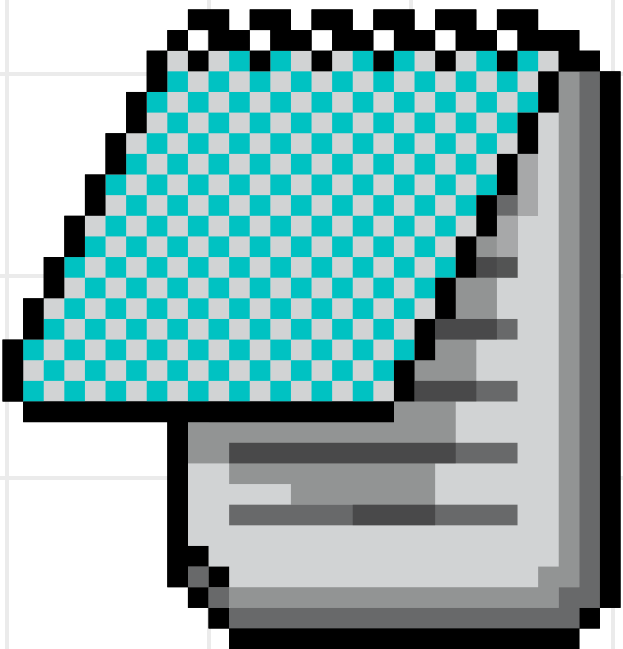


```
def heap_sort(arr, mostrar=False):
    n = len(arr)
    # -----
    # 1. Construir Max Heap
    # -----
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    if mostrar:
        print("\n2. Max Heap construido:")
        print(arr)
    # -----
    # 2. Extraer elementos
    # -----
    if mostrar:
        print("\n3. Estado del arreglo después de cada extracción:")
    for i in range(n - 1, 0, -1):
        # Intercambiar la raíz con el último elemento
        arr[0], arr[i] = arr[i], arr[0]
        if mostrar:
            print(f"Extracción: {arr}")
        # Restaurar el Max Heap
        heapify(arr, i, 0)
    # -----
    # 3. Lista final
    # -----
    if mostrar:
        print("\n4. Lista final ordenada:")
        print(arr)
```

SIGUIENTES EXTRACCIONES

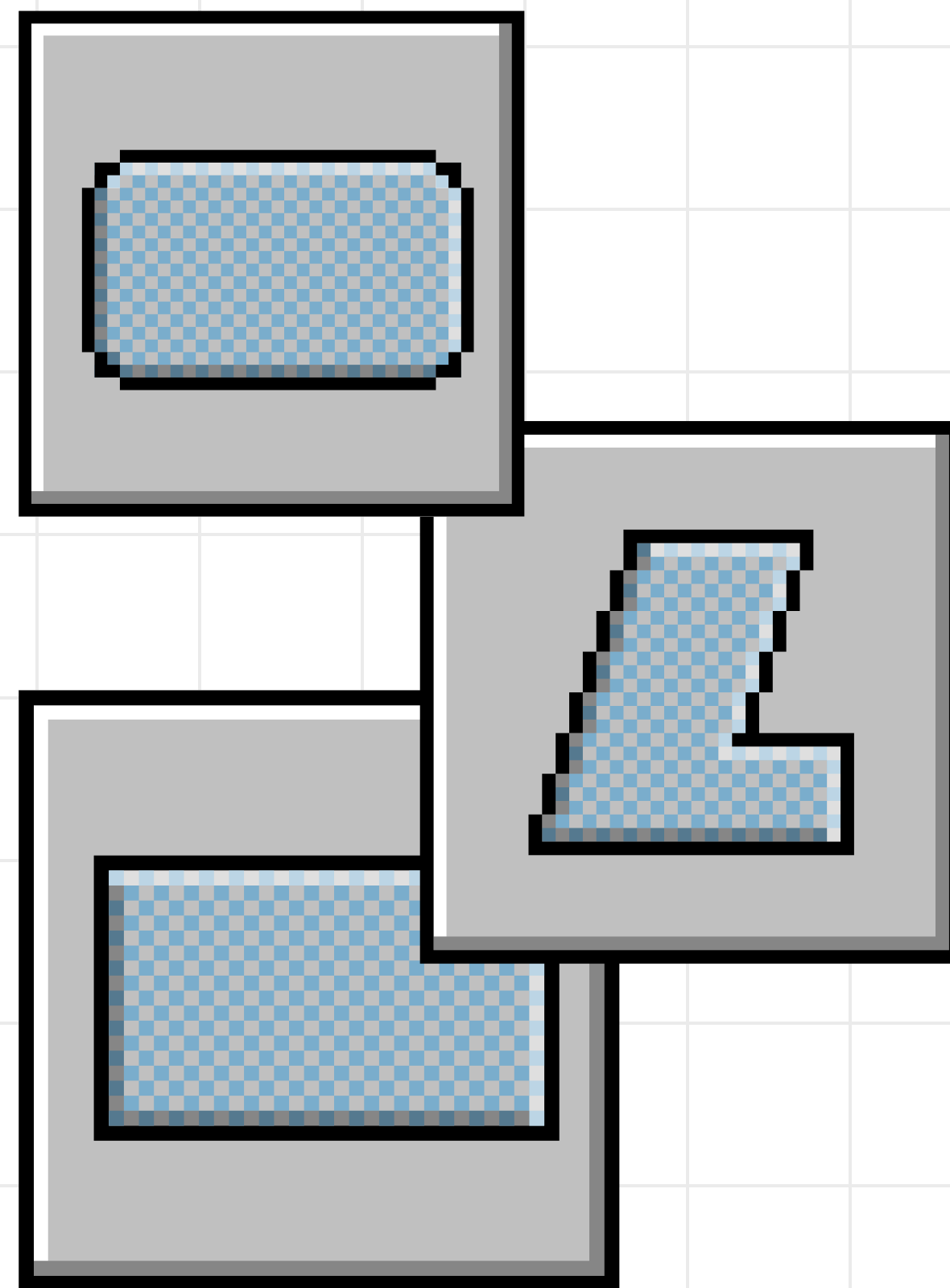


Cada extracción coloca el máximo al final

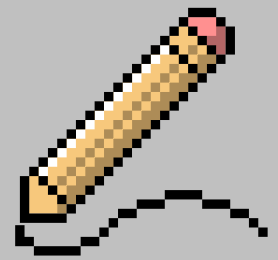
Extracción	Estado de extracción	Estado de la lista
Max Heap	[13, 11, 12, 5, 6, 7]	[13, 11, 12, 5, 6, 7]
1	[7, 11, 12, 5, 6, 13]	[12, 11, 7, 5, 6, 13]
2	[6, 11, 7, 5, 12, 13]	[11, 6, 7, 5, 12, 13]
3	[5, 6, 7, 11, 12, 13]	[7, 6, 5, 11, 12, 13]
4	[5, 6, 7, 11, 12, 13]	[6, 5, 7, 11, 12, 13]
5	[5, 6, 7, 11, 12, 13]	[5, 6, 7, 11, 12, 13]

RESULTADO FINAL

4. Lista final ordenada:
[5, 6, 7, 11, 12, 13]



CONCLUSIONES



¿Qué hace heapify0?

heapify() restaura la propiedad del Max Heap después de realizar un intercambio.

Compara:

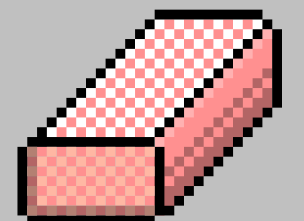
Padre

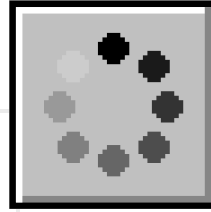


Hijo Hijo

Complejidad

Operacion	Complejidad
Construccion Heap	$O(n)$
Cada heapify	$O(\log n)$
Heap Sort	$O(n \log n)$





MEDICION DE TIEMPOS

===== MEDICIÓN DE TIEMPOS =====

Cantidad de elementos: 100

Tiempo de ejecución: 0.00010020 segundos

Cantidad de elementos: 500

Tiempo de ejecución: 0.00072080 segundos

Cantidad de elementos: 1000

Tiempo de ejecución: 0.00154270 segundos

Cantidad de elementos: 5000

Tiempo de ejecución: 0.01024490 segundos

Resultados:

Algoritmo	Aleatoria	Ordenada	Invertida
Heap Sort	0.00161370	0.00179270	0.00151920



GRACIAS!!!!

